SOFTWARE REQUIREMENTS SPECIFICATION

Loan & EMI Planner

B.Tech Full-Stack College Project

Next.js Full-Stack Application with Gemini AI

Document	Software Requirements Specification (SRS)
Project	Loan & EMI Planner
Application Type	Full-Stack Web Application
Frontend / Backend	Next.js
AI Service	Google Gemini API
Version	1.0
Status	Initial Requirements Specification

Purpose: Define the functional, technical, architectural, security, and non-functional requirements required to develop the Loan & EMI Planner.

Table of Contents

1. Introduction
2. Objectives
3. Scope
4. Stakeholders
5. User Roles
6. Functional Requirements
7. System Modules
8. Detailed Module Description
9. System Workflow
10. Use Cases
11. Database Requirements
12. Database Relationships
13. API Requirements
14. Frontend Requirements
15. Backend Requirements
16. Authentication and Authorization
17. Validation Requirements
18. Error Handling Requirements
19. Security Requirements
20. Non-Functional Requirements
21. Technology Stack
22. Hardware Requirements
23. Software Requirements
24. Assumptions
25. Constraints
26. Limitations
27. Future Scope
28. Acceptance Criteria
29. Project Completion Criteria
30. Core Architectural Principle

1. Introduction

1.1 Purpose

The purpose of the Loan & EMI Planner is to provide a simple, beginner-friendly web application through which users can calculate, understand, compare, and plan different types of loans. The system calculates financial values using reliable mathematical formulas and application logic and uses Google Gemini primarily to explain and interpret those calculated results in simple language.

1.2 Project Overview

The application provides EMI calculation, total interest and repayment calculation, complete amortization schedules, loan comparison, prepayment simulation, prepayment strategy planning, education loan planning, vehicle loan planning, and Gemini-based explanations.

1.3 Problem Statement

Many EMI calculators show only a monthly EMI and do not clearly explain the total financial impact of a loan. Users may not understand the interest component, principal repayment, outstanding balance, effect of prepayment, or differences between loan options. The proposed system addresses these issues by presenting complete loan-planning information in an understandable format.

1.4 Need for the System

- Simplify loan calculations.
- Improve understanding of EMI-based loans.
- Display complete repayment schedules.
- Help users compare loan options.
- Demonstrate the financial impact of prepayments.
- Help users plan additional repayments.
- Provide specialized education and vehicle loan planning.
- Explain financial results using simple natural language.

2. Objectives

- Calculate EMI accurately using standard mathematical formulas.
- Calculate total interest and total repayment amount.
- Generate a complete amortization schedule.
- Allow users to compare two or three loans.
- Simulate one-time loan prepayments.
- Calculate interest and tenure savings resulting from prepayment.
- Provide a monthly prepayment strategy planner.
- Provide education loan planning functionality.
- Provide vehicle loan planning functionality.
- Use Gemini AI to explain calculated results in simple language.
- Provide a clean, responsive, beginner-friendly interface.
- Maintain a simple architecture suitable for a B.Tech project.
- Make the application easy to understand, maintain, test, and explain during a viva.

3. Scope

3.1 In Scope

- EMI calculator and loan summary.
- Total interest and total repayment calculation.
- Complete amortization table.
- Loan comparison for two or three loans.
- One-time prepayment simulation.
- Prepayment strategy planning.
- Education loan planner.
- Vehicle loan planner.
- Gemini AI explanation.
- Input validation and error handling.
- Responsive user interface.
- Next.js frontend and server-side APIs.
- Reusable financial calculation services.

3.2 Out of Scope

- Actual loan application submission.
- Bank account integration or banking APIs.
- Credit score calculation or retrieval.
- Online loan approval.
- Payment processing or banking transactions.
- Investment, insurance, cryptocurrency, or portfolio management.
- Microservices, Kubernetes, complex DevOps, or enterprise infrastructure.
- Complex authentication or user management.
- Unnecessary AI/ML functionality.

4. Stakeholders

Stakeholder	Interest
End User	Calculate and understand loans.
Project Developer	Develop and maintain the system.
College Faculty	Evaluate functionality and architecture.
Project Guide	Review project progress.
Future Developer / AI	Continue or maintain the project.

5. User Roles

5.1 Guest / User

The initial version does not require mandatory registration. The user can calculate EMI, view amortization, compare loans, simulate prepayment, use the strategy planner, use education and vehicle loan planners, and request Gemini explanations. No administrative role is required in Version 1.0.

6. Functional Requirements

FR-01: EMI Calculator

Description: Calculate monthly EMI from loan amount, annual interest rate, and tenure.

Input: Loan amount, annual interest rate, tenure.

Processing: Convert annual rate to monthly rate and apply the standard reducing-balance EMI formula.

Output: Monthly EMI, total principal, total interest, total repayment.

Validation: Amount > 0; interest rate >= 0; tenure > 0; valid numeric inputs.

FR-02: Amortization Table

Description: Generate a month-by-month repayment schedule.

Input: Calculated loan parameters.

Processing: For each period calculate interest, principal and closing balance.

Output: Payment number, opening balance, EMI, interest, principal, closing balance.

Validation: Final balance should be approximately zero subject to rounding.

FR-03: Loan Comparison

Description: Compare two or three loan options.

Input: Loan amount, rate, tenure for each loan.

Processing: Calculate each loan independently using the same calculation engine.

Output: EMI, total interest, total payment, tenure and principal comparison.

Validation: Each supplied loan must be valid.

FR-04: Prepayment Simulation

Description: Simulate a one-time additional payment.

Input: Original loan, prepayment amount and prepayment timing.

Processing: Generate original schedule, identify outstanding principal, apply prepayment, recalculate remaining schedule, and compare scenarios.

Output: New balance, interest saved, original/new tenure, time saved.

Validation: Prepayment >= 0 and not greater than applicable outstanding principal; timing must be valid.

FR-05: Prepayment Strategy Planner

Description: Simulate regular additional payments.

Input: Existing loan and additional monthly payment.

Processing: Simulate repayment using original EMI plus the additional payment.

Output: Effective payment, revised tenure, interest saved, time saved.

Validation: Additional payment must be valid and non-negative.

FR-06: Education Loan Planner

Description: Provide specialized education-loan planning.

Input: Loan amount, rate, study period, moratorium, repayment tenure.

Processing: Calculate according to clearly documented project assumptions.

Output: EMI, interest, repayment, relevant schedule and moratorium information.

Validation: Inputs must be valid; lender-specific policies are not claimed.

FR-07: Vehicle Loan Planner

Description: Plan vehicle financing.

Input: Vehicle price, down payment, rate, tenure.

Processing: Loan amount = vehicle price - down payment, followed by standard loan calculation.

Output: Vehicle price, down payment, loan amount, EMI, interest, repayment and schedule.

Validation: Down payment must be non-negative and not exceed vehicle price.

FR-08: Gemini AI Explanation

Description: Explain already-calculated financial results in simple language.

Input: Validated calculated results and explanation context.

Processing: Send relevant calculated values to Gemini through a secure server-side route.

Output: Plain-language explanation.

Validation: Gemini is not the numerical source of truth; API errors must be handled.

EMI Formula

$$EMI = [P \times r \times (1+r)^n] / [(1+r)^n - 1]$$

Where P is principal, r is the monthly interest rate, and n is the number of monthly payments.

Amortization calculations follow: Interest = Opening Balance × Monthly Rate; Principal = EMI − Interest; Closing Balance = Opening Balance − Principal.

7. System Modules

- User Interface Module
- Loan Input and Validation Module
- Financial Calculation Module
- EMI Calculator Module
- Amortization Module
- Loan Comparison Module
- Prepayment Simulation Module
- Prepayment Strategy Module
- Education Loan Module
- Vehicle Loan Module
- Gemini AI Explanation Module
- API Module
- Error Handling Module

8. Detailed Module Description

User Interface Module

Navigation, forms, results, tables, charts/visual summaries, responsive layouts, loading states and error messages.

Financial Calculation Module

Reusable business logic for EMI, interest, total payment, amortization, remaining balance, prepayment and strategy simulation. It must not depend on Gemini.

Comparison Module

Processes multiple loan scenarios and presents consistent side-by-side results.

AI Explanation Module

Prepares calculated results for Gemini, sends requests through a secure server-side API, receives the explanation and displays it.

9. System Workflow

User opens application → selects planner → enters loan details → input validation → financial calculation engine → calculated results → results displayed → optional AI explanation → Next.js server API → Gemini API → explanation displayed.

10. Use Cases

UC-01: Calculate EMI

- User enters loan amount, rate and tenure.
- System validates inputs.
- System calculates EMI, total interest and total payment.
- System displays the loan summary.

UC-02: View Amortization

- User calculates a loan.
- System generates the amortization data.
- User views the repayment table.

UC-03: Compare Loans

- User enters two or three loan configurations.
- System calculates each loan.
- System displays side-by-side results.

UC-04: Simulate Prepayment

- User enters loan details and prepayment information.
- System calculates original and revised scenarios.
- System displays interest and tenure savings.

UC-05: Request AI Explanation

- User calculates a loan.
- User requests an explanation.
- Server sends calculated results to Gemini.
- Gemini returns an explanation.
- Application displays it.

11. Database Requirements

A persistent database is not required for Version 1.0. The current requirements do not include user accounts, saved plans, calculation history, admin data, or financial transactions. Calculations can be performed during the active session without persistent storage.

12. Database Relationships

Not applicable to Version 1.0 because no persistent application database is required. If persistence is introduced later, the database design must be documented before implementation.

13. API Requirements

API-01: AI Explanation

Property	Requirement
Method	POST
Endpoint	/api/ai/explain
Purpose	Generate a natural-language explanation of calculated loan results.
Request Body	Loan information, calculated financial results, and explanation context.
Response	Explanation text / structured explanation response.
Authentication	No user authentication in Version 1.0; Gemini credential remains server-side.

Core financial calculations may remain local application/server logic instead of being exposed as separate HTTP endpoints. This avoids unnecessary APIs for simple deterministic functions.

14. Frontend Requirements

Home Page

Purpose: Introduce the application and provide navigation to calculators and planners.

Components/Data: Header, navigation, hero section, feature cards, planner links.

EMI Calculator

Purpose: Calculate loan repayment information.

Components/Data: Loan form, summary, total interest, total payment, visualization, amortization table, AI explanation.

Loan Comparison

Purpose: Compare two or three loan options.

Components/Data: Loan A/B/C forms, comparison table/cards, AI explanation.

Prepayment Planner

Purpose: Understand the effect of a one-time prepayment.

Components/Data: Existing loan form, prepayment inputs, before/after comparison, savings summary.

Prepayment Strategy

Purpose: Understand regular additional payments.

Components/Data: Existing loan inputs, additional payment, revised tenure, savings summary.

Education Loan Planner

Purpose: Provide education-specific planning.

Components/Data: Education loan form, study period, moratorium, repayment information, summary, schedule.

Vehicle Loan Planner

Purpose: Calculate vehicle financing.

Components/Data: Vehicle price, down payment, rate, tenure, loan amount, EMI, interest, payment and schedule.

15. Backend Requirements

Next.js will act as both the frontend framework and backend platform.

Server responsibilities include handling API requests, server-side validation, protecting the Gemini API key, communicating with Gemini, returning structured responses, and handling API errors.

Financial calculations shall be implemented as reusable modules independent of UI components.

16. Authentication and Authorization

Authentication is not required for Version 1.0 because the current project does not require user accounts, saved calculations, personal profiles, or admin functionality. Avoiding authentication keeps the project focused and beginner-friendly.

17. Validation Requirements

- Loan amount must be greater than zero.
- Interest rate must be zero or greater.
- Tenure must be greater than zero.
- Vehicle down payment must be non-negative and must not exceed vehicle price.
- Prepayment must be non-negative and must not exceed the applicable outstanding principal.
- Numeric values must be valid numbers.
- Required fields must not be empty.

18. Error Handling Requirements

- Handle invalid or missing input.
- Handle invalid numeric values.

- Handle API failures and Gemini errors.
- Handle unexpected server errors.
- Handle financial calculation edge cases.
- Show clear user-facing messages without exposing internal server details.

Example messages:

Please enter a valid loan amount.
Interest rate cannot be negative.
Please enter a valid tenure.
Unable to generate AI explanation. Please try again.

19. Security Requirements

- Gemini API credentials must remain server-side.
- API keys must be stored in environment variables.
- Secrets must never be committed to Git.
- User inputs must be validated on both relevant client and server boundaries.
- Internal errors must not be unnecessarily exposed.
- Only necessary information should be sent to Gemini.
- AI-generated explanations must not be presented as official financial advice.

20. Non-Functional Requirements

Requirement	Description
Performance	Normal calculations should execute quickly; large schedules should be handled efficiently; Gemini requests should show loading states.
Usability	The interface should be simple, readable, consistent and beginner-friendly.
Reliability	The same valid input should produce the same financial calculation result. Financial calculations must not depend on Gemini.
Maintainability	Use modular components, separate UI and calculation logic, meaningful names, minimal duplication and readable code.
Security	Sensitive API credentials remain server-side.
Responsiveness	The application should work on desktop, tablet and mobile.
Compatibility	Support modern browsers such as Chrome, Edge, Firefox and Safari.

21. Technology Stack

Technology	Purpose
Next.js	Full-stack application framework.
React	UI development through Next.js.
TypeScript	Type-safe development.
Google Gemini API	AI-based explanations.
CSS / Tailwind CSS	Responsive UI styling.
Database	Not required in Version 1.0.

22. Hardware Requirements

- Minimum: 4 GB RAM, dual-core processor, approximately 5 GB available storage, and internet connectivity.
- Recommended: 8 GB or more RAM, modern multi-core processor, and stable internet.

23. Software Requirements

- Node.js
- npm, pnpm, or equivalent package manager
- Git
- Modern web browser
- Visual Studio Code or another code editor
- Gemini API access/key
- Operating system capable of running Node.js

24. Assumptions

- Users provide valid loan information.
- Calculations use a standard monthly reducing-balance EMI model.
- Annual interest rates are converted into monthly rates.
- Loan tenure is represented in months internally.
- Financial values may be rounded for display.
- Actual bank/lender calculations may differ.
- Education-loan policies may differ by lender.
- Prepayment rules may differ between lenders.
- Gemini is an explanation layer rather than numerical authority.
- The application is an educational/planning tool and not a replacement for official lender calculations.

25. Constraints

- The project is intended for a B.Tech college project.
- Architecture must remain beginner-friendly.
- Unnecessary infrastructure should be avoided.
- Gemini availability depends on API access.
- Internet connectivity is required for AI explanations.
- The project does not guarantee lender-specific calculations.
- Version 1.0 does not provide transactions or loan approval.

26. Limitations

- No direct bank connectivity.
- No live lender interest rates.
- No loan application submission.
- No guarantee of lender-specific prepayment calculations.
- AI explanations may be imperfect and are explanatory rather than authoritative.
- No permanent storage of calculations in the initial version.
- No regulated financial advice.

27. Future Scope

- User accounts and saved loan plans.
- Calculation history.
- Database persistence.
- PDF/CSV export.
- Additional loan types.
- Real-time lender rate comparison.
- Advanced prepayment strategies.
- Notifications/reminders.
- Multi-language explanations.
- Enhanced charts.
- Deployment analytics.

28. Acceptance Criteria

- User can enter valid loan details.
- EMI is calculated correctly.
- Total interest and repayment are calculated correctly.
- Amortization schedule is generated.
- Principal and interest are shown for each repayment period.
- Two loans can be compared and a third can be optionally added.
- Prepayment simulation produces before/after results.
- Interest and tenure savings are displayed.
- Regular additional payment can be simulated.
- Education loan planning works according to documented assumptions.
- Vehicle loan planning works.
- Gemini can explain calculated results.
- Gemini credentials remain secure.
- Invalid input is handled properly.
- The application is responsive.
- Major functionality works without depending on Gemini.
- Project functionality can be demonstrated and explained during a viva.

29. Project Completion Criteria

- All core modules are implemented.
- Financial calculations are tested and verified.
- Amortization calculations are verified.
- Loan comparison works.
- Prepayment simulation works.
- Prepayment strategy works.
- Education loan planner works.
- Vehicle loan planner works.
- Gemini explanation works.
- Frontend and backend are integrated.
- Validation and error handling are implemented.

- Responsive UI is completed.
- Documentation reflects the actual implementation.
- Known issues are documented.
- The application can be demonstrated successfully.

30. Core Architectural Principle

The central design rule of the project is that the application performs all authoritative financial calculations, while Gemini explains those results.

```
USER INPUT → INPUT VALIDATION → FINANCIAL CALCULATION ENGINE → VERIFIED RESULTS → UI DISPLAY / GEMINI EXPLANATION
```

Gemini must never be treated as the source of truth for EMI, interest, amortization, or prepayment calculations. This separation improves reliability, testability, security, and explainability.

End of Software Requirements Specification